Rest Assured API Automation Framework

A developer's guide to how this framework was built, and how it works

Java 17 | Maven | Rest Assured 5.4 | TestNG | Allure | GitHub Actions

Target under test: automationexercise.com/api_list — 14 REST endpoints

Repository: github.com/nikhiljsupekar/Restassured_API_Framework

Contents

1. What is Rest Assured, and why this stack
2. Project architecture at a glance
3. The build, step by step
4. The given / when / then DSL, explained
5. Walking through the five test classes
6. Chaining stateful tests with TestNG
7. Two real bugs found while validating against the live API
8. Understanding the Allure report
9. The CI/CD pipeline
10. Running it yourself
11. Rest Assured cheat sheet
12. Extending the framework

1. What is Rest Assured, and why this stack

Rest Assured is a Java library that gives HTTP API testing a readable, fluent syntax (`given() / when() / then()`) instead of hand-rolling `HttpURLConnection` or Apache HttpClient calls. It handles request building, response parsing (JSON/XML), and assertion chaining in one DSL, which is why it's the de-facto standard for Java-based API test automation.

This project pairs it with:

Tool	Role	Why it was chosen
Maven	Build tool	Most common companion for Rest Assured; simple dependency management via `pom.xml`.
TestNG	Test runner	Supports `@BeforeClass` / `@AfterClass` lifecycle hooks and `dependsOnMethods` — both used here to model stateful API workflows (e.g. create → read → update → delete).
Allure	Reporting	Turns raw test output into a browsable, shareable HTML report with full request/response detail per test — useful for showing non-technical stakeholders that things pass.

GitHub Actions	CI/CD	Runs the suite on every push and auto-publishes the Allure report to GitHub Pages, so there's always a current, link-shareable result.

2. Project architecture at a glance

The project follows a standard Maven layout: production-style helper code lives under `src/main/java`, and the actual test classes live under `src/test/java`. Nothing in `main` is a "test" — it's the reusable plumbing that every test class depends on.

```
Restassured_API_Framework/
├── pom.xml                      # dependencies + build/report plugins
├── testng.xml                   # which test classes run, in what mode
├── .github/workflows/           # CI pipeline
├── src/main/java/com/automationexercise/api/
│   ├── config/ConfigManager.java      # reads config.properties (base URL, timeouts)
│   ├── constants/Endpoints.java       # endpoint path constants, one per API
│   ├── base/BaseTest.java             # shared RequestSpecification + filters
│   ├── pojo/UserPayload.java          # typed builder for account form fields
│   └── utils/
│       ├── TestDataFactory.java       # builds valid test users w/ unique emails
│       └── ApiAssertions.java         # shared responseCode/message assertions
└── src/test/java/com/automationexercise/api/tests/
    ├── ProductsApiTest.java       # APIs 1-2
    ├── BrandsApiTest.java         # APIs 3-4
    ├── SearchProductApiTest.java  # APIs 5-6
    ├── LoginApiTest.java          # APIs 7-10
    └── AccountApiTest.java        # APIs 11-14
```

Every test class extends `BaseTest`, which means every test automatically gets a pre-configured `RequestSpecification` pointed at the right base URL, with logging and Allure reporting already wired in. That's the core idea of this kind of framework: **write the plumbing once, reuse it everywhere.**

3. The build, step by step

This is the order the framework was actually assembled in, and why each piece exists before the next one needed it.

Step 1 — `pom.xml`: declare the stack

Before any Java code, the dependencies had to exist: `rest-assured`, `testng`, `allure-testng` + `allure-rest-assured` (the Allure/Rest-Assured bridge that captures request/response as

report attachments), `jackson-databind`, and a logging backend (`slf4j-simple`). The `maven-surefire-plugin` is configured to run `testng.xml` as its suite file, and the `allure-maven` plugin turns raw Allure JSON results into a static HTML report on demand.

Step 2 — `config.properties` + `ConfigManager`

The base URL (`https://automationexercise.com`) is never hardcoded in a test. It lives in one properties file, read once by a small static utility:

```
public static String baseUri() {
    return System.getProperty("base.uri", PROPERTIES.getProperty("base.uri"));
}
```

The `System.getProperty` fallback means you can override the target environment from the command line without touching a file: `mvn test "-Dbase.uri=https://staging.automationexercise.com"`.

Step 3 — `Endpoints`: name every path once

A plain constants class maps each of the 14 API paths to a named field (`PRODUCTS_LIST`, `VERIFY_LOGIN`, etc.). This is the difference between a test failing with `"/api/verifyLogin"` typo'd three different ways across three files, versus one compile error in one place.

Step 4 — `BaseTest`: the shared request spec

Every API test needs the same base setup: a base URI, sane timeouts, and something capturing the request/response for the report. Rather than repeating that in every test class, `BaseTest` builds it once in a TestNG `@BeforeClass` hook:

```
@BeforeClass(alwaysRun = true)
public void setUpBase() {
    requestSpec = new RequestSpecBuilder()
            .setBaseUri(ConfigManager.baseUri())
            .setConfig(config)
            .addFilter(new AllureRestAssured())
            .addFilter(new RequestLoggingFilter())
            .addFilter(new ResponseLoggingFilter())
            .build();
}
```

Every test class extends `BaseTest` and simply uses `requestSpec` — it never rebuilds this itself.

Step 5 — `UserPayload` + `TestDataFactory`

The account endpoints (`createAccount` , `updateAccount`) take 17 form fields. `UserPayload` is a typed builder for that shape, with a `toFormParams()` method that turns it into the `Map<String, Object>` Rest Assured's `.formParams( ... )` expects. `TestDataFactory` then produces a *valid, ready-to-use* user with a timestamp-unique email, so two test runs never collide on the same account:

```
public static String uniqueEmail() {
    return "qa.user." + System.currentTimeMillis() + "@example.com";
}
```

Step 6 — the test classes themselves

With the plumbing in place, each test class is just: build a request off `requestSpec` , call the endpoint, assert the result. See section 5 for a full walkthrough.

Step 7 — `testng.xml` : wire the suite together

Lists which test classes run and in what mode. It currently runs **sequentially** (`parallel="none"`) — see section 7 for why that specific decision mattered.

4. The given / when / then DSL, explained

Every Rest Assured call in this project follows the same three-part shape. It reads almost like a sentence:

```
Response response = given(requestSpec)              // GIVEN: starting conditions
        .formParam("search_product", "top")  //   (headers, params, body ... )
        .when()                                        // WHEN: the action
        .post(Endpoints.SEARCH_PRODUCT)
        .then()                                         // THEN: the expectation
        .statusCode(200)
        .extract().response();                          // pull out the Response object
```

Block	Purpose	Common calls used here
`given()`	Set up the request	`formParam()` , `formParams()` , `queryParam()`
`when()`	Fire the HTTP call	`get()` , `post()` , `put()` , `delete()`

`then()`	Validate + extract	`statusCode()`, `extract().response()`

Note this framework **doesn't** chain assertions inside `.then()` via `.body( ... )` — see the bug write-up in section 7 for exactly why.

5. Walking through the five test classes

ProductsApiTest & BrandsApiTest (APIs 1–4)

The simplest pattern in the suite: one happy-path GET, one unsupported-method call that the API deliberately rejects. automationexercise.com always answers with HTTP 200 and puts the real result in a JSON `responseCode` field — so `postToProductsList_returnsMethodNotSupported()` still asserts `statusCode(200)` at the HTTP layer, then separately checks that the JSON body says `responseCode: 405`.

SearchProductApiTest (APIs 5–6)

Same shape, but exercises a required form parameter: sending `search_product=top` returns matches; omitting it returns `responseCode: 400` with a specific error message. Both are asserted, not just the code — a wrong error message on a 400 is still a bug.

LoginApiTest (APIs 7–10)

`verifyLogin` only makes sense against an account that already exists. Rather than depending on another test class's leftover data, this class is self-contained: it creates a throwaway user in `@BeforeClass` and deletes it in `@AfterClass`, regardless of whether the tests in between pass or fail (`alwaysRun = true` on the cleanup). The four tests then cover: valid login (200), missing email (400), unsupported DELETE verb (405), and a login attempt against a non-existent email (404).

AccountApiTest (APIs 11–14)

This is the one true lifecycle test: create an account, fetch it by email, update it, then delete it. Each step only makes sense if the previous one succeeded, so they're chained with TestNG's `dependsOnMethods` — covered in detail in the next section.

6. Chaining stateful tests with TestNG

Most API test suites are independent — each test can run alone, in any order. The account lifecycle can't: you cannot fetch, update, or delete a user that was never created. `AccountApiTest` makes that dependency explicit instead of implicit:

```
@Test(description = "API 11: createAccount")
public void createAccount_returnsCreated() { ... }

@Test(dependsOnMethods = "createAccount_returnsCreated")
public void getUserDetailByEmail_returnsUser() { ... }

@Test(dependsOnMethods = "getUserDetailByEmail_returnsUser")
public void updateAccount_returnsOk() { ... }

@Test(dependsOnMethods = "updateAccount_returnsOk", alwaysRun = true)
public void deleteAccount_returnsOk() { ... }
```

> **Why this matters**
>
> If `createAccount` fails, TestNG automatically *skips* the three downstream tests instead of running them and producing three confusing, unrelated failures. `alwaysRun = true` on the delete step is a deliberate exception: even if an earlier step fails, we still attempt cleanup so a failed run doesn't leave orphaned test accounts on the live site.

7. Two real bugs found while validating against the live API

The framework compiled and looked correct on paper — but running it against the actual live site surfaced two real issues. Both are worth understanding because they're common traps, not specific to this project.

Bug 1 — `.body(path, matcher)` silently mis-parses the response

automationexercise.com returns every response with `Content-Type: text/html`, even though the body is JSON. Rest Assured's `.then().body("responseCode", equalTo(200))` shorthand decides whether to parse the body as JSON or XML *based on that header* — and once a `ResponseLoggingFilter` had touched the response stream (to print it to the console), that detection broke completely, producing a bizarre "XML path doesn't match" error where the "actual" value was the entire raw JSON string.

Fix:

stop relying on content-type auto-detection. Every assertion in this project instead extracts the response and reads it with `response.jsonPath().getInt("responseCode")` explicitly — which always parses as JSON regardless of what the header claims. That logic lives in one place, `ApiAssertions`, so no test has to know about this quirk directly.

Bug 2 — sporadic HTTP 520 errors under parallel execution

The first version of `testng.xml` ran test classes in parallel (`parallel="classes"`, 3 threads). Against a real production site fronted by Cloudflare, that concurrency triggered intermittent `520 Unknown Error` responses — not a bug in the framework, but the site rate-limiting/throttling a public demo endpoint.

Fix:

`testng.xml` now runs sequentially (`parallel="none"`). It's slower, but reliable — the right trade-off when the system under test is a shared public site you don't control, rather than a dedicated test environment.

Takeaway

Both bugs only showed up when the suite actually ran against the live API — neither was visible from reading the code. This is the core argument for never treating "it compiles" as "it works": always run an API framework against the real target before trusting it.

8. Understanding the Allure report

Every test run writes raw result files to `target/allure-results` (one JSON file per test, plus attachments for each HTTP request/response, thanks to the `AllureRestAssured` filter in `BaseTest`). Those raw files aren't human-readable on their own — the Allure tooling turns them into a browsable report with:

Section	What it shows
Overview / Dashboard	Pass/fail counts, duration, trend graph across runs.
Suites	Every test class and method, with pass/fail status per test.
Behaviors	Grouped by the `@Epic` / `@Feature` / `@Story` annotations on each test — e.g. all "Account Lifecycle" tests together.
Test detail	Click into any test to see the exact request sent and response received, step by step — this is the part worth showing a client directly.

Two ways to view it:

```
# Live local server, opens your browser automatically
mvn allure:serve

# Static HTML into target/site/allure-maven-plugin (needs a real web
# server to view, not a double-clicked file — Allure fetches its
# data via AJAX, which file:// URLs block)
mvn allure:report
```

9. The CI/CD pipeline

`.github/workflows/api-tests.yml` runs on every push/PR to `main`. The pipeline has two jobs:

1. `test` — checks out the repo, installs JDK 17, runs `mvn clean test` against the live API, generates the Allure report (`mvn allure:report`), and uploads it as a Pages artifact plus the raw Surefire XML as a build artifact.
2. `deploy` — takes that Pages artifact and publishes it via `actions/deploy-pages`, straight to GitHub Pages. No `gh-pages` branch is involved; it's a native Pages deployment.

> **One-time setup required**
>
> For the `deploy` job to work, the repository's
> **Settings → Pages → Source**
> must be set to
> **"GitHub Actions"**
> (not "Deploy from a branch"). Without that, the workflow succeeds but has nowhere to publish to.

The result: every push produces a fresh, link-shareable report at `https://<username>.github.io/<repo>/` — no manual report generation or file-sharing needed.

10. Running it yourself

Prerequisite	Notes
JDK 17+	Compiles down via `maven.compiler.release=17`, so a newer installed JDK (e.g. 25) works fine.
Maven 3.9+	See the README's "Prerequisites" section for a working install path — `winget install Apache.Maven` does not resolve to a real package.

```
# run the full suite
mvn clean test

# override the target environment for one run
mvn clean test "-Dbase.uri=https://staging.automationexercise.com"

# view the report
mvn allure:serve
```

11. Rest Assured cheat sheet

Syntax	What it does
`given(requestSpec)`	Start a request from the shared base spec.
`.formParam(k, v)` / `.formParams(map)`	Add `application/x-www-form-urlencoded` fields — what this API expects for POST/PUT bodies.
`.queryParam(k, v)`	Add a `?key=value` query string param (used for the GET-by-email lookup).
`.when().get/post/put/delete(path)`	Fire the HTTP call with the given verb.
`.then().statusCode(200)`	Assert the raw HTTP status line.
`.extract().response()`	Pull out a reusable `Response` object for further inspection.

`response.jsonPath().getInt("field")`	Read a JSON field explicitly — the reliable way used throughout this project (see Bug 1).
`response.jsonPath().getList("products")`	Read a JSON array field as a Java `List`.
`@BeforeClass` / `@AfterClass`	TestNG hooks for one-time setup/teardown per test class.
`dependsOnMethods = "x"`	Skip this test automatically if the named test failed.

12. Extending the framework

To add coverage for a new endpoint:

1. Add its path as a constant in `Endpoints.java`.
2. Add a `@Test` method to the relevant existing test class, or a new class extending `BaseTest` if it's a new feature area.
3. Use `ApiAssertions.assertResponseCode( ... )` / `assertMessage( ... )` for the standard checks, and `response.jsonPath()` directly for anything more specific.
4. Register the new class in `testng.xml` if it's a new file.

Everything downstream — Allure reporting, CI execution, the published report — picks up new tests automatically with no other changes needed.
